UNIP
UNIVERSIDADE PAULISTA

7º/8º Ciência da Computação (CC)

Orientações para a disciplina de

Atividades Práticas Supervisionadas

2026/2

- TEMA
- PROPOSTA DO TRABALHO
- APRESENTAÇÃO DO TRABALHO

Atividades Práticas Supervisionadas (APS)

I. TEMA:

"ENGENHARIA DE ESCALABILIDADE HORIZONTAL VS. VERTICAL SOB RESTRIÇÃO DE RECURSOS"

II. PROPOSTA DO TRABALHO

As Atividades Práticas Supervisionadas serão constituídas da seguinte forma:

A cadeia de observação ambiental

Uma infraestrutura analítica de monitoramento microclimático e prevenção de catástrofes ambientais opera sobre uma cadeia de observação distribuída geograficamente. Centros de monitoramento climático mantêm redes de estações de coleta instaladas em território urbano e periurbano, cada uma equipada com sensores que medem grandezas atmosféricas elementares. As grandezas coletadas são a temperatura do ar, expressa em graus Celsius, e a umidade relativa, expressa em percentual. A combinação das duas é o insumo de um índice de calor percebido, calculado a partir de uma formulação não linear que estima a pressão de vapor e corrige o valor pela umidade.

O índice adotado é o Humidex. Sua formulação envolve uma exponencial sobre a temperatura e uma divisão em ponto flutuante, o que o caracteriza como operação intensiva em aritmética e, portanto, limitada por CPU. A decisão de calcular o índice no servidor central, e não no dispositivo de campo, decorre de restrição energética: os sensores operam com bateria e captação solar, e o orçamento de energia disponível não comporta esse tipo de processamento em regime contínuo. O dispositivo de borda mede e transmite; o servidor central calcula, persiste e publica.

A cadeia completa compreende, portanto, quatro elos: a aquisição da grandeza no sensor, o transporte pela camada de comunicação, a ingestão e o processamento no servidor central, e o consumo do dado agregado pelos sistemas de automação que tomam decisão. Cada elo impõe restrições próprias ao projeto, e

é a interação entre elas que define o problema computacional tratado nesta atividade.

Os sensores de borda e os hubs de concentração

A topologia da rede é hierárquica em dois níveis. No nível mais baixo situam-se os sensores de borda, dispositivos de baixo custo e baixo consumo, responsáveis pela aquisição das grandezas e pelo armazenamento temporário local. No nível intermediário situam-se os hubs de concentração, equipamentos que agregam a telemetria de um conjunto de sensores de uma mesma região e a encaminham ao servidor central.

A função do hub é determinante para o comportamento observado no servidor. Ele não se limita a repassar dados: acumula as leituras recebidas dos sensores durante um intervalo e as descarrega em bloco quando o enlace com o servidor central está disponível. Esse mecanismo de buffer existe porque a conectividade em campo é intermitente — o enlace oscila com a cobertura, e em áreas de relevo acidentado ou vegetação densa pode permanecer indisponível por períodos prolongados.

O resultado desse desenho é que o servidor central não observa um fluxo uniforme de leituras individuais, mas lotes de tamanho variável. Cada lote contém várias leituras do mesmo sensor, espaçadas no tempo e terminando no instante da descarga. O tamanho do lote é função direta do tempo em que o hub permaneceu sem enlace, o que introduz uma variabilidade que não pode ser eliminada por

A camada de comunicação e o regime de transmissão

A comunicação entre os níveis da rede emprega protocolos celulares de quinta geração voltados a dispositivos massivos de baixa potência, especificamente NB-IoT e LTE-M. Essas tecnologias foram projetadas para o padrão de tráfego característico da Internet das Coisas: transmissões curtas, intervalos longos de inatividade e consumo energético reduzido. Suas características impõem três propriedades ao tráfego que alcança o servidor central: a taxa de transmissão é baixa por dispositivo, a latência é variável e a disponibilidade do enlace não é garantida.

A consequência agregada dessas propriedades é a natureza em rajadas do tráfego de ingestão. Em regime estável, os hubs descarregam seus buffers em ritmo previsível, e o servidor observa um fluxo basal aproximadamente constante. Sob

determinadas condições atmosféricas — inversão térmica, chegada de frente fria, elevação abrupta de temperatura —, um grande número de hubs passa a descarregar simultaneamente, e o volume que alcança o servidor em poucos segundos excede em ordens de grandeza o fluxo nominal.

O sistema é, portanto, submetido a um regime de demanda variável, no qual um fluxo basal constante é sobreposto por rajadas periódicas de pico. A relevância dessa caracterização é que as rajadas não constituem anomalia: são o comportamento esperado da rede precisamente nos momentos em que o dado tem maior valor operacional. Um serviço dimensionado apenas para o regime basal não atende ao propósito da infraestrutura.

A ingestão no servidor central

O servidor central expõe uma interface de ingestão que recebe lotes de medições por requisição. Cada medição transporta o identificador do sensor, o instante da leitura, as grandezas microclimáticas e o pacote bruto de telemetria recebido do dispositivo. O processamento de cada lote compreende três etapas sequenciais: o cálculo do índice Humidex para cada medição, a gravação transacional do conjunto e a resposta ao hub solicitante.

A resposta ao hub é o ponto em que a arquitetura síncrona se torna crítica. Enquanto a resposta não é emitida, a conexão permanece aberta e o hub aguarda confirmação. Como o custo de processamento de um lote é proporcional ao seu tamanho — cada medição exige o cálculo do índice —, lotes grandes ocupam a conexão por mais tempo, e a concorrência de muitos hubs produz acúmulo de conexões pendentes.

A esse custo computacional soma-se o custo de persistência. A gravação de um lote em transação única, com confirmação de escrita no disco, impõe uma operação de entrada e saída que não é paralelizável dentro da mesma transação. O tempo total de atendimento de uma requisição de ingestão é, assim, a soma de uma parcela limitada por CPU e de uma parcela limitada por entrada e saída, ambas crescendo com o tamanho do lote.

O consumo por sistemas de automação

Os dados persistidos não são consumidos por operadores humanos em consulta individual. São consumidos por sistemas de automação que avaliam continuamente

o estado do território e tomam decisões a partir de limiares predefinidos. Esses sistemas consultam o índice de calor médio por sensor e por região, e não as leituras individuais: a agregação é a grandeza que sustenta a decisão.

A distinção entre as duas operações define a distribuição de custo do sistema. A escrita é limitada por CPU, em razão do cálculo do índice e da confirmação transacional. A leitura é limitada por volume, porque a média precisa ser calculada sobre um conjunto de linhas que cresce continuamente ao longo da operação. Uma solução que otimize apenas um dos lados não altera o comportamento global, e é por isso que o projeto arquitetural precisa considerar as duas rotas em conjunto.

Acrescente-se que o padrão de acesso é assimétrico. O número de consultas supera substancialmente o número de ingestões, porque cada hub envia um lote esporádico enquanto múltiplos sistemas de automação consultam continuamente. Essa assimetria é característica de infraestruturas de monitoramento e deve ser refletida na topologia adotada.

A integridade do registro

O registro de medições possui função probatória. Decisões operacionais e administrativas tomadas a partir do índice de calor precisam ser justificáveis posteriormente, o que exige que cada medição seja verificável de forma independente. Para atender a esse requisito, cada medição recebe, no momento da ingestão, uma âncora criptográfica calculada sobre o identificador do sensor, o instante canônico da leitura e o pacote bruto recebido.

A âncora é calculada por medição, e não por lote nem por sensor, porque cada leitura constitui evidência autônoma. A função de resumo empregada garante que qualquer alteração em qualquer dos três campos de entrada produza um resultado completamente distinto, tornando a adulteração detectável. A verificação é feita por recálculo: o valor armazenado é confrontado com o valor recalculado a partir dos campos devolvidos pela interface de consulta.

Dessa exigência decorre uma restrição adicional ao projeto. A âncora depende do instante da leitura em forma canônica, o que significa que a representação temporal do dado não é livre: qualquer variação de formato produz um resumo diferente para a mesma medição. O contrato da aplicação, detalhado na Seção IV,

fixa essa representação e estabelece a unicidade do par formado pelo identificador do sensor e pelo instante da leitura.

A tolerância temporal da operação

A operação não exige que uma medição esteja disponível para consulta imediatamente após a emissão do sensor. A janela de tolerância é de dez minutos, e decorre do ciclo de avaliação dos sistemas de automação, que operam sobre consolidados periódicos e não sobre leituras isoladas.

Essa premissa tem consequência arquitetural direta: autoriza a separação entre o instante da resposta ao hub e o instante da persistência efetiva. O sistema pode confirmar o recebimento antes de concluir a gravação, desde que a gravação se complete dentro da janela. Em termos de modelo de consistência, admite-se consistência eventual, e é essa admissão que viabiliza as topologias assíncronas discutidas na Seção V.

O que a avaliação mensura não é a latência da resposta, mas a preservação integral do dado. Uma medição emitida que não alcança o armazenamento constitui perda permanente e irreversível, e no contexto de um registro com função probatória representa também uma lacuna de observação. A janela de dez minutos existe para que a equipe possa trocar latência por vazão sem ser penalizada por essa escolha.

III. O SERVIÇO E A ARQUITETURA DISPONÍVEL

O serviço que atende essa cadeia de observação é um monolito síncrono implementado na linguagem Go, apoiado em um banco de dados PostgreSQL. Em regime nominal, o desenho atende ao propósito: a rota de ingestão recebe o lote, calcula o índice de cada medição, grava o conjunto em uma única transação e responde ao hub; a rota de consulta calcula a média por sensor diretamente no banco. O dimensionamento, no entanto, foi estabelecido para o regime nominal, e não para o regime de pico.

Os fatores de colapso

Três características do monolito se combinam para produzir o colapso sob a rajada.

A primeira é o processamento síncrono. Cada requisição de ingestão precisa calcular o Humidex de todas as medições do lote antes de responder. Como o

cálculo é intensivo em CPU e o lote pode conter até 60 medições, o custo de uma requisição é elevado e proporcional ao tamanho do lote.

A segunda é o pool de conexões limitado. A aplicação mantém no máximo dez conexões simultâneas com o banco. Quando mais de dez requisições precisam gravar ao mesmo tempo, as excedentes aguardam na fila interna da aplicação, o que converte a contenção de banco em latência de resposta.

A terceira é a cota de CPU. A aplicação dispõe de 5% de uma vCPU. Com essa fração, o cálculo do índice de um lote grande consome tempo perceptível, e a fila de requisições cresce mais rapidamente do que a aplicação consegue drená-la.

O resultado é uma cascata: a fila interna cresce, as conexões permanecem ocupadas por mais tempo, o kernel acumula conexões pendentes, e o gerador de carga, que injeta requisições a uma taxa fixa, passa a ter iterações descartadas. O sistema não se torna apenas lento: ele deixa de absorver a demanda.

O orçamento de recursos

O monolito opera sob um orçamento fixo e imutável. A aplicação recebe 0.05 de vCPU e 64 MB de memória, com o runtime travado em duas thread de sistema (GOMAXPROCS=2) e o pool de conexões limitado a dez. O banco recebe 0.08 de vCPU e 256 MB.

Esse orçamento constitui a restrição física que define o problema. Não há possibilidade de resolver a rajada por ampliação de CPU, porque não há CPU adicional disponível. O que existe é um volume fixo de ciclos computacionais, e a única variável sob controle da equipe é a forma de distribuí-los entre os componentes da solução.

IV. O PROJETO DE ESCALABILIDADE

A tarefa consiste em reprojetar a topologia do serviço para que ele absorva a rajada sem perda de dados, mantendo o mesmo orçamento agregado de recursos. O instrumento é a escalabilidade horizontal combinada com técnicas de sistemas distribuídos: múltiplas instâncias da aplicação, balanceamento de carga, filas de mensagens, caches, bancos de apoio e processamento assíncrono.

O contrato HTTP da aplicação não pode ser alterado. O comportamento observável da API é fixo e constitui a interface de avaliação: a equipe pode

reorganizar tudo o que se encontra atrás da porta 8080, mas as rotas, os formatos de requisição e de resposta e as regras de identidade do dado permanecem como estão

A matriz comparativa contém exatamente dois cenários. O Cenário A é o baseline de referência, fixo e imutável, fornecido pelo docente. O Cenário B é a arquitetura distribuída, de desenho inteiramente livre pela equipe, submetida no arquivo complementar. Não existem cenários intermediários predefinidos: qualquer topologia que a equipe queira comparar internamente é decisão de projeto do grupo e deve ser documentada no relatório, mas a matriz oficial é composta apenas pelas duas linhas a seguir.

Tabela 1 — Cenários da matriz

Cenário	Configuração Arquitetural	Hipótese Analítica a Verificar
Cenário A (Baseline)	Monolito síncrono unitário fornecido pelo docente — aplicação em 0.05 CPU + PostgreSQL em 0.08 CPU, GOMAXPROCS=2, MaxConns=10	Colapso da fila síncrona por saturação do pool de conexões e do kernel, com descarte de iterações sob a rajada de pico.
Cenário B (Distribuído)	Arquitetura distribuída de desenho livre pela equipe, submetida em docker-compose.distribuido.yaml, com cota individual de 0.05 CPU por instância da aplicação e soma de limits.cpus limitada a 0.80	Isolamento do cliente HTTP na borda com absorção assíncrona da rajada de dados; ganho de vazão e de integridade sob o mesmo orçamento agregado de CPU.

Duas regras fixam o experimento. A primeira é a cota individual: cada instância da aplicação declarada no arquivo distribuído usa exatamente 0.05 de vCPU, a mesma do baseline. O que a equipe decide é quantas instâncias existem e como elas se organizam, nunca a cota de cada uma. Fixar a cota individual é o que isola a arquitetura como variável independente: se cada instância recebe sempre a mesma fração de CPU, a diferença de resultado entre dois desenhos decorre exclusivamente da forma de distribuição do trabalho, e não de instâncias com cotas distintas.

A segunda é o teto agregado: a soma de todas as frações de processamento declaradas em limits.cpus não pode ultrapassar 0.80, ou seja, 80% de uma vCPU. Com a cota individual fixada em 0.05, esse teto comporta no máximo 16 instâncias da aplicação, descontando o que a solução alocar a banco de dados, fila, cache e proxy reverso. A questão que a bancada responde é: concedido um volume fixo de

ciclos computacionais, qual organização topológica extrai a maior eficiência útil de trabalho?

Os entregáveis

A equipe deve submeter três artefatos.

Primeiro, o projeto arquitetural em Docker Compose: o arquivo complementar docker-compose.distribuido.yaml, que sobrepõe a arquitetura original. A porta TCP de entrada deve ser mantida obrigatoriamente como 8080.

Segundo, a matriz comparativa de testes: registro tabular da vazão alcançada em requisições por segundo, da taxa percentual de sucesso de processamento e do Índice de Eficiência de Vazão, para os dois cenários, coletando adicionalmente métricas de contenção e uso de recursos através do utilitário docker stats durante todas as janelas ativas de estresse.

Terceiro, o relatório técnico estruturado: documento formal em formato PDF que responda às indagações, com a matriz preenchida e capturas do docker stats.

Requisitos de contrato da solução

Além dos três artefatos, a solução deve satisfazer os requisitos abaixo, cujo descumprimento invalida a auditoria.

O serviço de entrada deve ser publicado na rede Docker aps-net com o alias de rede aps-app, de modo que a plataforma de teste o alcance em http://aps-app:8080. O alias não é sugestão de nomenclatura: é o endereço fixo usado pelos componentes de carga e de auditoria.

Nenhum volume declarado no compose pode ser do tipo external, uma vez que dados remanescentes de uma execução anterior contaminariam a contagem de registros da execução seguinte.

A aplicação deve expor as rotas descritas na Seção VI, com resposta em JSON e sem exigência de autenticação, já que o tráfego ocorre em rede Docker isolada.

Regulamento técnico

As diretrizes a seguir constituem o regulamento mandatário da atividade.

Imutabilidade do núcleo base: os arquivos main.go, verificar_registros.go, avaliar_justo.go, script.js, Dockerfile.monolito, docker-compose.monolito.yaml, docker-compose.teste.yaml, run_benchmark.bat, run_benchmark.sh e init.sql não podem sofrer qualquer alteração estrutural ou de valores.

Cota individual fixa: toda instância da aplicação declarada no arquivo distribuído deve usar exatamente cpus: '0.05'. Instâncias com cota diferente, para mais ou para menos, invalidam a comparação e são tratadas como infração.

Isolamento de código: toda e qualquer extensão de engenharia, criação de workers, introdução de proxies reversos ou bancos de dados intermediários deve constar estritamente dentro do arquivo complementar docker-compose.distribuido.yaml.

Permissões tecnológicas: é facultado ao grupo o emprego livre de tecnologias auxiliares de alto desempenho dentro da composição, incluindo Nginx, HAProxy, RabbitMQ, Apache Kafka, Redis, instâncias de apoio em Go, além de bancos relacionais ou NoSQL, desde que suportados pela infraestrutura Docker local.

Infrações desclassificatórias: a violação do teto unificado de 0.80 CPU no somatório de todos os serviços do arquivo distribuído, a alteração da cota individual de 0.05, a alteração da porta pública 8080, a quebra do contrato de auditoria ou do alias aps-app, a adoção de mockups que descartem a carga computacional sem seu respectivo cálculo ou adulterações diretas nos arquivos de métricas report.json e stream.json implicam atribuição imediata de nota zero (0,0) por fraude técnica.

V. O CONTRATO DA APLICAÇÃO

Requisitos funcionais

A aplicação precisa oferecer cinco operações, todas em HTTP e com resposta em JSON.

A primeira é a verificação de prontidão: a rota GET /health responde HTTP 200 quando a aplicação está apta a receber tráfego, e HTTP 503 quando o banco está indisponível. É por essa rota que a plataforma de teste decide quando começar a injetar carga.

A segunda é a ingestão de telemetria: a rota POST /telemetria recebe um lote de medições e o persiste. Cada medição traz o identificador do sensor, o instante da

leitura, as grandezas microclimáticas e o pacote bruto de telemetria. A aplicação calcula o índice Humidex de cada medição do lote e grava o conjunto.

A terceira é a consulta do índice de calor: a rota GET /indice-calor/{sensor_id} devolve o índice médio calculado para aquele sensor, junto com o número de amostras consideradas. Quando o sensor ainda não possui medição registrada, a resposta é HTTP 404, o que é um resultado legítimo do contrato e não uma falha.

A quarta e a quinta são as rotas de auditoria, descritas na Subseção 6.3, que informam os totais persistidos e listam as medições gravadas de forma paginada.

O contrato de ingestão

A rota POST /telemetria recebe um lote de medições por requisição, tipicamente entre 10 e 60 unidades. O corpo da requisição é um objeto com o campo measurements, um array em que cada item contém sensor_id, timestamp, metrics.microclimate e raw_telemetry_packet.

O timestamp é a identidade temporal da medição e obedece a uma forma canônica obrigatória: UTC com exatamente seis casas decimais, no formato AAAA-MM-DDTHH:MM:SS.ffffffZ. Essa exigência não é estética. O timestamp participa do cálculo do hash, e qualquer variação de representação, como três casas decimais em vez de seis, fuso local em vez de UTC ou sufixo omitido, produz hash diferente para a mesma medição, invalidando o cruzamento. O banco pode armazenar o instante em qualquer tipo nativo, inclusive com precisão superior; o que se exige é que a rota de listagem devolva o timestamp serializado na forma canônica. A coluna criado_em, por convenção, registra o instante de recebimento pelo servidor e não substitui o timestamp do evento: são grandezas distintas, e apenas o timestamp do evento compõe a identidade da medição.

Cada medição carrega uma âncora criptográfica própria, calculada por medição e não por lote nem por sensor:

hash = SHA-256(sensor_id + timestamp_canônico + raw_packet), em hexadecimal minúsculo.

A concatenação é direta, sem separadores. A ausência de separador é deliberada: como sensor_id tem tamanho fixo e o timestamp tem tamanho fixo, não existe ambiguidade de fronteira entre os campos. A função de avalanche do SHA-

256 garante que uma diferença mínima no timestamp, de um microssegundo, produza um hash completamente distinto, e é essa propriedade que permite usar o hash como identidade inequívoca da medição no cruzamento entre o que foi despachado e o que foi persistido.

A unicidade é exigida sobre o par sensor_id e timestamp. Duas medições do mesmo sensor podem transportar o mesmo raw_packet em instantes distintos, o que representa a reamostragem de um valor estável dentro de um intervalo curto; nesse caso os hashes serão necessariamente diferentes, pois o timestamp diverge. O que não pode ocorrer é a persistência de duas linhas com o mesmo sensor_id e o mesmo timestamp.

A resposta da ingestão deve informar quantas medições foram recebidas, quantas foram efetivamente persistidas e quantas foram rejeitadas por invalidade. Essa granularidade importa para a interpretação das métricas: um lote aceito com HTTP 2xx pode ter persistido menos medições do que recebeu, e por isso a taxa de sucesso de requisições não é equivalente à taxa de persistência de registros. As duas grandezas são medidas separadamente

Os exemplos de chamada desta rota, com requisição e resposta completas, estão no Anexo A.

O contrato de consulta e de auditoria

A consulta do índice de calor é feita pela rota GET /indice-calor/{sensor_id}, que devolve o índice médio do sensor e o número de amostras. É essa rota que os sistemas de automação consomem

A auditoria é conduzida por duas rotas. A rota GET /audit/contagem informa os totais persistidos, com os campos total_medicoes e total_indices. A rota GET /audit/medidas lista as medições gravadas de forma paginada, com os campos id, sensor_id, timestamp, raw_packet e hash em cada item, mais o campo proximo_cursor.

A paginação é por cursor, e não por deslocamento, porque permanece consistente mesmo sob inserção concorrente: o cursor ancora a posição em um registro específico, e não em um índice ordinal que se desloca a cada nova linha. O valor do cursor é opaco, e a plataforma de avaliação não o interpreta, apenas o devolve na requisição seguinte. A ausência do cursor, campo vazio ou nulo, sinaliza

o fim do conjunto. A aplicação pode escolher qualquer critério de ordenação estável, desde que o cursor avance e que o conjunto completo seja alcançável por iteração sucessiva. O valor padrão de limit é 1000.

O princípio que sustenta essas rotas é o de propriedade do dado: a aplicação é a única proprietária do banco de dados, e nenhum componente da plataforma de avaliação abre conexão com o PostgreSQL, seja para leitura de contagem, seja para inspeção de registros. Toda verificação de persistência passa pelas rotas HTTP. Isso produz três consequências de projeto. A primeira é o desacoplamento entre a avaliação e o esquema físico: o aluno pode renomear tabelas, normalizar, particionar, replicar ou substituir o mecanismo de armazenamento sem que a auditoria quebre, desde que preserve o contrato. A segunda é a impossibilidade de leitura de réplica desatualizada, problema intrínseco ao acesso direto em topologias com replicação. A terceira é a simetria: a aplicação é auditada exatamente pelo mesmo caminho pelo qual é consumida em produção, de modo que qualquer gargalo introduzido na camada de leitura também é medido.

O cumprimento integral desse contrato é condição necessária para a avaliação. Ausência de uma rota, resposta fora do formato especificado, paginação que não avança ou hash calculado sobre campos diferentes dos declarados resultam em auditoria inválida, independentemente do desempenho observado na matriz comparativa. Os exemplos de chamada de cada rota estão no Anexo A, e a lista de verificação está no Anexo C.

Restrições não funcionais

Os requisitos não funcionais definem as condições em que a aplicação opera e são, nesta atividade, tão importantes quanto os funcionais.

O primeiro é a cota de processamento. Cada instância da aplicação recebe exatamente 0.05 de vCPU, e a soma de todas as frações de processamento da solução não pode ultrapassar 0.8. Essa restrição é o coração do exercício: o orçamento de hardware é fixo e imutável, e o que se compara é a organização topológica, não a quantidade de recursos.

O segundo é o limite de tempo de resposta. A aplicação tem um teto de tempo para responder a cada requisição, e requisições que o excedem são contabilizadas como perdidas. Esse teto é o que impede que uma solução

simplesmente enfileire tudo internamente e responda devagar: a resposta precisa sair dentro da janela, ainda que a persistência ocorra depois.

O terceiro é a porta de entrada, fixada em 8080, e o alias de rede aps-app na rede aps-net, que é o endereço pelo qual a plataforma de teste alcança a aplicação. O quarto é a ausência de volumes externos O quinto é a janela de atualização dos dados, em que não há exigência de que uma medição esteja consultável imediatamente após a emissão. A tolerância é de 30 segundos.

VI. A CARGA DE TESTE

Não é um teste sintético de laço fechado, em que um número fixo de clientes repete requisições o mais rápido que consegue. É um teste de taxa de chegada: a plataforma define quantas requisições por segundo devem chegar, e as injeta nesse ritmo, independentemente de a aplicação conseguir acompanhar. Se a aplicação não absorve a taxa, as iterações excedentes são descartadas e contabilizadas como tal.

Essa escolha é deliberada. Em um teste de laço fechado, uma aplicação lenta simplesmente recebe menos carga, e o problema fica invisível. Em um teste de taxa de chegada, a carga é imposta de fora, e a incapacidade de absorver aparece como descarte. É o que permite distinguir uma solução que responde rápido de uma solução que responde devagar.

O fluxo basal

A carga tem dois regimes sobrepostos, e ambos são configuráveis por variáveis de ambiente, sem alteração do código.

O primeiro é o fluxo basal, constante do início ao fim da execução. Ele é composto por duas correntes independentes: uma corrente de escrita, que envia lotes de telemetria a uma taxa WRITE_RATE, e uma corrente de leitura, que consulta o índice de calor a uma taxa READ_RATE. As leituras predominam sobre as escritas, refletindo a assimetria de acesso em que múltiplos sistemas de automação consultam continuamente enquanto relativamente poucos hubs escrevem.

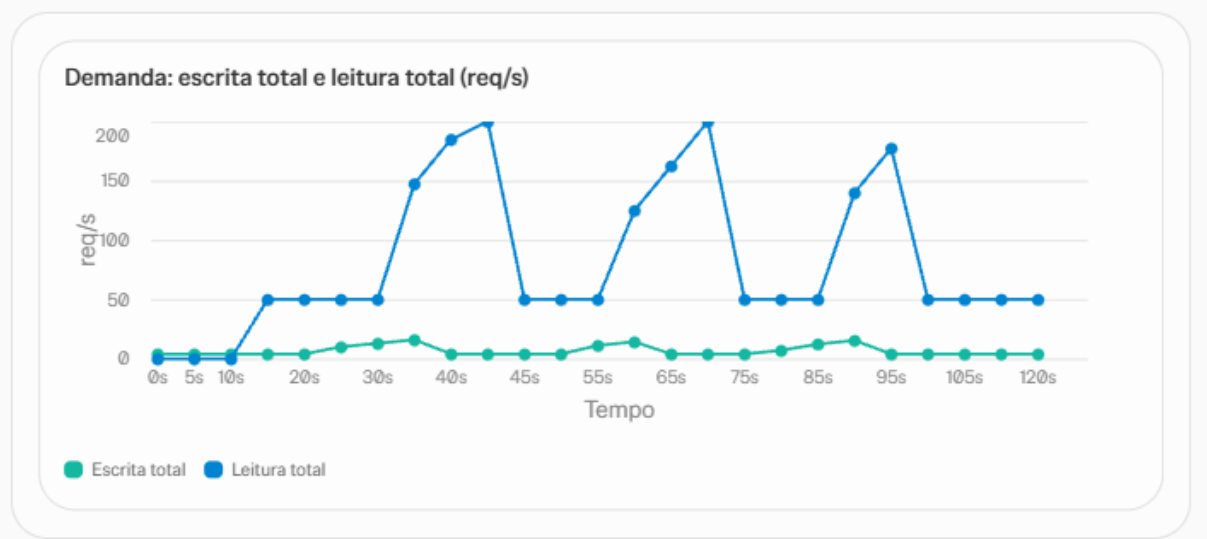
As rajadas de pico

O segundo regime são as rajadas de pico, que reproduzem o comportamento descrito na Subseção 3.3. A partir do segundo PEAK_START, três picos consecutivos são sobrepostos ao fluxo basal, tanto na escrita quanto na leitura.

Cada pico tem a mesma forma: rampa de subida de 2 segundos, patamar crítico de 10 segundos e corte de 2 segundos, com vales de 14 segundos entre um pico e o seguinte. Durante o patamar, as taxas de escrita e de leitura são multiplicadas pelo fator PEAK_FACTOR.

A forma da rajada é intencional. A rampa curta não dá tempo de a aplicação se preparar. O patamar sustenta a pressão por tempo suficiente para saturar os recursos. O vale testa se a aplicação se recupera ou permanece degradada. E a repetição em três picos verifica se a solução resiste a eventos sucessivos, e não apenas a um.

Figura 1 – Curva de demandas de leitura e escrita



O padrão dos lotes

A corrente de escrita não envia medições isoladas. Cada iteração envia um lote de BATCH_MIN a BATCH_MAX medições do mesmo sensor, com o tamanho sorteado aleatoriamente dentro dessa faixa. Esse padrão representa o buffer de hub descrito na Subseção 3.2.

O padrão de lote tem duas consequências que precisam ser compreendidas. A primeira é que o número de medições cresce muito acima do número de requisições: um lote de 60 medições é uma única requisição HTTP. Por isso a

contagem de registros e a contagem de requisições são grandezas distintas, e a nota é calculada sobre a primeira. A segunda é que o custo de processamento de uma requisição de ingestão é proporcional ao tamanho do lote, e não constante: cada medição do lote exige o cálculo do Humidex. Uma requisição com 60 medições custa muito mais CPU do que uma com 10.

As medições de um mesmo lote são espaçadas de um minuto entre si e terminam no instante do envio, reproduzindo a leitura bufferizada. O identificador do sensor segue o padrão SNS-5G-SEQ-NNNNNN, e a corrente de leitura consulta sensores de um pool pré-criado, de modo que parte das consultas retorna 404, o que é resposta válida e não conta como falha.

VII. AS MÉTRICAS E A NOTA

Como a execução é medida

A execução é dividida em duas fases sequenciais e isoladas. A primeira sobe o sistema sob teste, aguarda a prontidão pela rota de saúde e executa o gerador de carga. A segunda só inicia após a primeira encerrar completamente e após a verificação de que o relatório de métricas foi produzido. A separação é estrutural, e não um artifício de temporização: o gerador escreve suas métricas ao término da execução, e o avaliador lê esse arquivo e consulta a API. Executar os dois em paralelo produziria leitura de arquivo incompleto e contagem prematura.

O gerador mantém um manifesto de tudo o que despachou, gravado como fluxo de eventos no arquivo stream.json. Para cada medição enviada, antes da requisição HTTP, ele registra o sensor_id e o hash calculado. O registro ocorre antes do envio de propósito: se a requisição estourar o timeout, a medição continua contabilizada como despachada, o que é correto, porque ela saiu do gerador e a perda, se houver, é responsabilidade da aplicação.

Como arquiteturas assíncronas respondem antes de persistir, a contagem é observada por amostragem repetida até que dois ciclos consecutivos apresentem os mesmos valores em total_medicoes e total_indices, por três leituras estáveis, ou até o esgotamento do prazo configurado. Não existe interrupção antecipada por comparação entre gravados e enviados: com ingestão em lote, a contagem de

medições cresce muito acima da contagem de requisições, e um critério de parada baseado nessa comparação cortaria a drenagem no meio. O prazo padrão é de 90 segundos e pode ser ampliado por variável de ambiente; recomenda-se 180 a 300 segundos para arquiteturas assíncronas, em que a persistência ocorre após a confirmação ao cliente.

Os contadores de integridade

O avaliador baixa o conjunto completo de medições pela rota paginada e cruza cada registro com o manifesto, produzindo quatro contadores.

Divergentes são os registros cujo hash armazenado não corresponde ao hash recalculado sobre sensor_id, timestamp e raw_packet. O avaliador não confia no hash devolvido pela API: ele recalcula e compara. Divergência indica erro de serialização ou corrupção.

Fantasmas são os registros presentes na API e ausentes do manifesto, ou seja, persistidos sem terem sido despachados.

Perdidas são as medições presentes no manifesto e ausentes da API, ou seja, despachadas e não persistidas.

Duplicatas são os pares sensor_id e timestamp que aparecem mais de uma vez, violando a regra de unicidade.

Há ainda um contador de diagnóstico, leitura_sem_dado, que registra quantas consultas retornaram 404 no decorrer do teste. Em uma solução síncrona, esse número tende a ser baixo e concentrado no início da execução. Em uma solução com índice assíncrono, pode permanecer alto durante toda a janela, sinalizando atraso entre a persistência da medição e a atualização do índice de calor. O dado é informativo e não compõe a nota, mas é exigido na discussão do relatório.

O cálculo da nota

A nota é proporcional à taxa de persistência de registros, conforme a fórmula:

nota = 10 × (gravados ÷ enviados)

A escolha da razão entre gravados e enviados, e não entre gravados e requisições aceitas, é deliberada. O que importa em um sistema de telemetria é a integridade do dado: uma medição que o sensor emitiu e que não chegou ao

armazenamento é uma perda permanente e irreversível. A aplicação pode responder com presteza, aceitar todos os lotes e ainda assim descartar registros; a nota deve refletir essa diferença.

Respostas perdidas por timeout entram no denominador mas não no numerador, e penalizam corretamente. Uma taxa superior a 100 por cento é tratada como anomalia e sinalizada no log: indica dado residual de execução anterior sobrevivendo entre rodadas, normalmente por volume externo ou container não removido.

O avaliador oficial roda em container isolado imediatamente após a finalização do k6 e da janela de sincronização. Ele desconsidera a latência temporal absoluta em milissegundos, para afastar discrepâncias de hardware físico entre processadores dos alunos, e concentra-se puramente na integridade e na vazão útil relativa entregue. A nota final é escalonada em régua linear entre 0,0 e 10,0 pontos e decompõe-se em dois pilares.

A nota final é multiplicativa:

Nota final = 10 × Efetivo × Integridade

Se qualquer pilar for nulo, a nota final é zero. Commits tardios são creditados pelo drain, mas não inflacionam a nota.

VIII. APRESENTAÇÃO DO TRABALHO

1. O grupo deverá ser composto de 5 alunos. A formação de um grupo com um número diferente de 5 dependerá de aprovação do(a) coordenador(a) auxiliar do curso no campus.

2. Todas as etapas do trabalho deverão ser escritas em fonte ARIAL 12, espaçamento 1,5, margem direita 2,5 cm e margem esquerda 2,5 cm. O trabalho deverá ter formato A4, encadernado (espiral) com capa transparente.

3. Limites de páginas
   - Objetivo do trabalho: 1 página e no máximo 2 páginas

- Introdução : 2 páginas e no máximo 4 páginas
- Fundamentos das tecnologias de escalabilidade e sistemas distribuídos escolhidas (conceitos gerais): 4 páginas e no máximo 8 páginas.
- Plano de desenvolvimento da aplicação: mínimo de 5 páginas e máximo de 15 páginas.
- Projeto (estrutura e módulos) da arquitetura distribuída desenvolvida: mínimo de 3 páginas e máximo de 8 páginas.
- Relatório com as linhas de código: máximo de 10 páginas.
- Apresentação do sistema em funcionamento em um computador, apresentando todas as funcionalidades pedidas e extras
- Bibliografia
- Ficha de Atividades Práticas Supervisionadas

4. O trabalho deverá ser entregue junto com a ficha padrão de “Atividades Práticas Supervisionadas” ilustrando cronologicamente cada um dos itens, segundo a orientação do professor supervisor desta atividade.

5. Estrutura do trabalho:

    1. Capa: identificando o curso, o tema, a relação de alunos do grupo (nome/RA)
    2. Índice
    3. Objetivo e motivação do trabalho
    4. Introdução
    5. Fundamentos das tecnologias de escalabilidade e sistemas distribuídos (conceitos gerais)
    6. Plano de desenvolvimento da aplicação (elementos e ferramentas que serão utilizadas)
    7. Projeto (estrutura e módulos que serão desenvolvidos) do programa
    8. Relatório com as linhas de código do programa
    9. Apresentação do programa em funcionamento em um computador, apresentando todas as funcionalidades pedidas e extras.
    10. Bibliografia
    11. Ficha de Atividades Práticas Supervisionadas

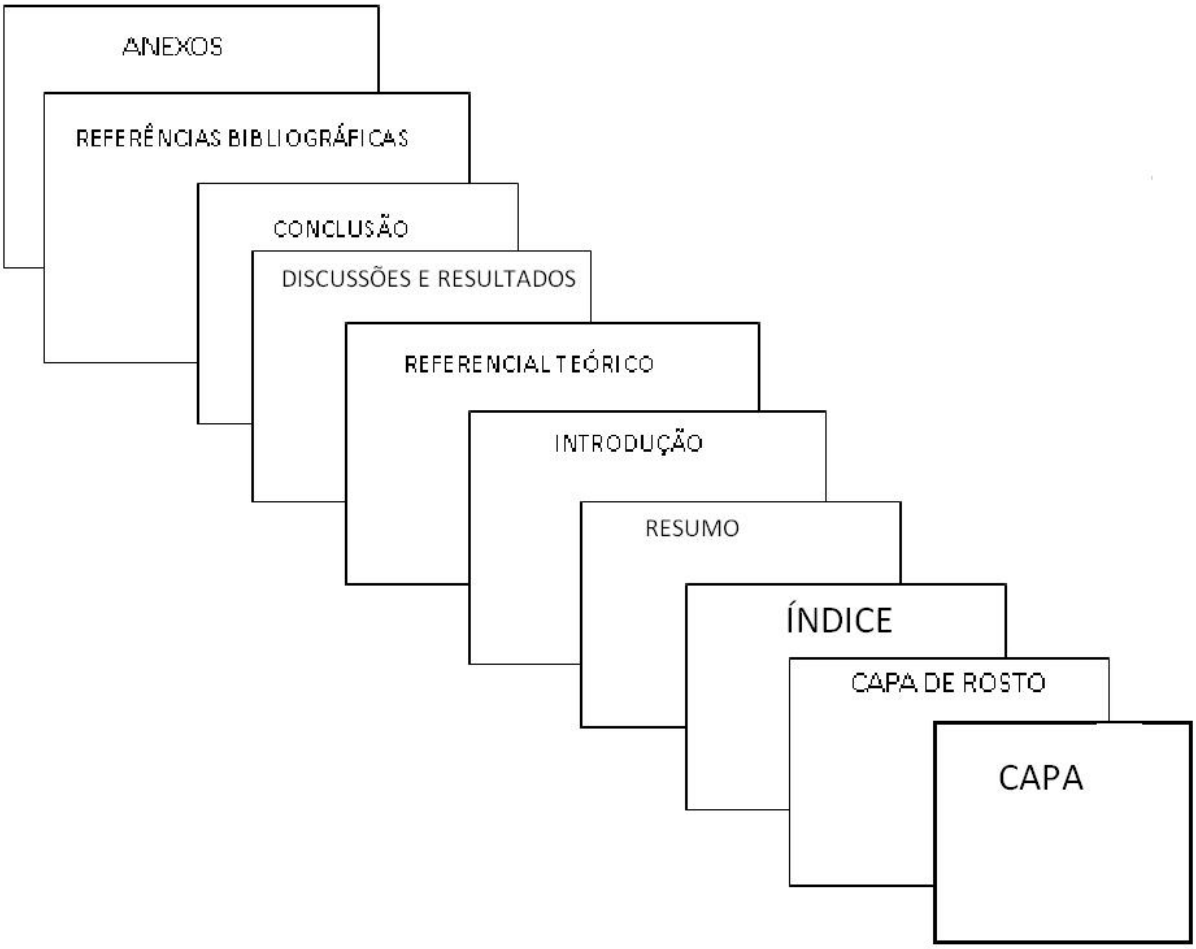
IMPORTANTE

SEM A POSTAGEM NO SISTEMA, DA PARTE TEÓRICA, É IMPOSSÍVEL LANÇAR A NOTA. SENDO ASSIM, MESMO QUE O GRUPO FAÇA A PARTE PRÁTICA, MAS NÃO POSTE NO SISTEMA, A PARTE TEÓRICA, A EQUIPE FICARÁ EM DEPENDÊNCIA (DP), NA DISCIPLINA.

SERÁ AVALIADO NO TRABALHO:

a) Construção Geral
b) Formatação
c) Normas da ABNT
d) Pontos destacados no Item “**PROPOSTA DO TRABALHO”.**
e) Discussões, Conclusões e Coerência.
f) Bibliografia
g) Montagem

MONTAGEM



<u>CÓPIAS DE TRABALHOS</u>: SERÃO DESCONSIDERADOS E EMITIDA NOTA ZERO, A QUALQUER TRABALHO QUE TENHA SIDO PLAGIADO DA INTERNET OU MESMO DA PRÓPRIA UNIP. VALE LEMBRAR QUE QUALQUER FRASE/TEXTO QUE VOCÊ UTILIZE EM SEU TRABALHO E, NÃO TENHA SIDO DESCRITA POR VOCÊ MESMO, MAS SIM POR OUTRA PESSOA, ESTA DEVERÁ SER DEVIDAMENTE REFERENCIADA CONFORME AS NORMAS DA ABNT.

PERCENTUAL DE PLÁGIO ATÉ 10%, IMPLICA EM TRABALHO APROVADO (ESTANDO OS DEMAIS ITENS DENTRO DO SOLICITADO)

PERCENTUAL DE PLÁGIO ACIMA DE 10%, IMPLICA EM TRABALHO REPROVADO.

IV. MODELO DE CAPA

Av. Alberto Benassi, 200 - Parque das Laranjeiras - Araraquara - SP
CEP 14804-300 - Tel.: (16) 3336-1800

ATIVIDADES PRÁTICAS SUPERVISIONADAS

EQUIPE

NOME	RA

Atividades Práticas Supervisionadas, apresentada à UNIP – Campus Araraquara/SP, referente ao 7º/8º Semestres, como parte dos requisitos para avaliação semestral, no Curso de Ciência da Computação.

ARARAQUARA/SP

2026

X. MODELO DE FICHA DE ATIVIDADES PRÁTICAS SUPERVISIONADAS

OBRIGATÓRIO ANEXAR A FICHA DA APS

NÃO HÁ A NECESSIDADE DE ASSINATURA, NA FICHA, POR PARTE DA COORDENAÇÃ

UNIVERSIDADE PAULISTA

FICHA DAS ATIVIDADES PRÁTICAS SUPERVISIONADAS - APS

NOME:_____ **TURMA:** _____ **RA:**_____

CURSO:_____ **CAMPUS:**_____ **SEMESTRE:**_____ **TURNO:**_____

CÓDIGO DA ATIVIDADE:_____ **SEMESTRE:**_____ **ANO GRADE:**_____

DATA DA ATIVIDADE	DESCRIÇÃO DA ATIVIDADE	TOTAL DE HORAS	ASSINATURA DO ALUNO	HORAS ATRIBUÍDAS (1)	ASSINATURA DO PROFESSOR

(1) Horas atribuídas de acordo com o regulamento das Atividades Práticas Supervisionadas do curso.

TOTAL DE HORAS ATRIBUÍDAS:_____

AVALIAÇÃO:_____

Aprovado ou Reprovado

NOTA:_____

DATA:_____/_____/_____

_____

CARIMBO E ASSINATURA DO COORDENADOR DO CURSO

ANEXO A — ENDPOINTS E EXEMPLOS DE CHAMADA

Tabela 2 — Contratos de requisição e resposta

Rota	Requisição	Resposta
GET /health	Sem corpo	200 com status de prontidão
POST /telemetria	Objeto com array measurements; cada item contém sensor_id, timestamp canônico, metrics.microclimate e raw_telemetry_packet	2xx com total_recebido, total_persistido e total_rejeitado
GET /indice-calor/{sensor_id}	Identificador do sensor no caminho	200 com índice calculado, ou 404 quando não há medição
GET /audit/contagem	Sem corpo	200 com total_medicoes e total_indices
GET /audit/medidas	Parâmetros limit e cursor na query string	200 com array medidas e proximo_cursor; cada item contém id, sensor_id, timestamp, raw_packet e hash

O array medidas deve ser ordenado de forma estável e a paginação deve percorrer o conjunto completo sem repetir nem omitir registros. O valor padrão de limit é 1000.

A.1 Exemplos de chamada com curl

Os exemplos usam a porta publicada no host (localhost:8080). Dentro da rede Docker, troque por http://aps-app:8080. Os valores de sensor, timestamp e hash são ilustrativos.

A.1.1 Prontidão da aplicação

```
===== CÓDIGO A.1.1 (curl GET /health) — INÍCIO =====
curl -i http://localhost:8080/health
===== CÓDIGO A.1.1 (curl GET /health) — FIM =====
```

Resposta esperada, com HTTP 200:

```
===== CÓDIGO A.1.1b (resposta GET /health) — INÍCIO =====
HTTP/1.1 200 OK
Content-Type: application/json

{"status":"ok"}
===== CÓDIGO A.1.1b (resposta GET /health) — FIM =====
```

Se o banco estiver indisponível, a rota responde HTTP 503 com o texto "Banco de dados indisponivel". A plataforma usa esta rota para saber quando a aplicação está pronta.

A.1.2 Ingestão de um lote de medições

```
===== CÓDIGO A.1.2 (curl POST /telemetria) — INÍCIO =====
curl -i -X POST http://localhost:8080/telemetria \
  -H "Content-Type: application/json" \
  -d '{
    "measurements": [
      {
        "sensor_id": "SNS-5G-SEQ-000001",
        "timestamp": "2026-10-23T14:32:05.123456Z",
        "metrics": {
          "microclimate": {
```

```
            "temperature_celsius": 27.4,
            "relative_humidity_percentage": 68.2
          }
        },
        "raw_telemetry_packet": "k3j9x2ab7c1"
      },
      {
        "sensor_id": "SNS-5G-SEQ-000001",
        "timestamp": "2026-10-23T14:33:05.123456Z",
        "metrics": {
          "microclimate": {
            "temperature_celsius": 27.9,
            "relative_humidity_percentage": 66.8
          }
        },
        "raw_telemetry_packet": "m8p2v4cd9e5"
      }
    ]
  }'
===== CÓDIGO A.1.2 (curl POST /telemetria) — FIM =====
```

Resposta esperada, com HTTP 202:

```
===== CÓDIGO A.1.2b (resposta POST /telemetria) — INÍCIO =====
HTTP/1.1 202 Accepted
Content-Type: application/json

{"status":"aceito","sensor_id":"SNS-5G-SEQ-000001","total_aceito":2,"humidex_medio":29.87}
===== CÓDIGO A.1.2b (resposta POST /telemetria) — FIM =====
```

Observações:

- O timestamp deve chegar em UTC com seis casas decimais. Outro formato produz hash diferente e o registro é contabilizado como divergente.
- O kit de referência devolve total_aceito, que no baseline síncrono equivale ao número de medições persistidas: o lote é gravado inteiro ou rejeitado inteiro. Soluções com rejeição parcial devem informar também total_persistido e total_rejeitado.
- Lote vazio responde HTTP 400 com "Lote de medidas vazio". JSON malformado responde HTTP 400 com "Payload de telemetria invalido".

A.1.3 Consulta do índice de calor

```
===== CÓDIGO A.1.3 (curl GET /indice-calor) — INÍCIO =====
curl -i http://localhost:8080/indice-calor/SNS-5G-SEQ-000001
===== CÓDIGO A.1.3 (curl GET /indice-calor) — FIM =====
```

Resposta esperada, com HTTP 200, quando o sensor possui medições:

```
===== CÓDIGO A.1.3b (resposta GET /indice-calor 200) — INÍCIO =====
HTTP/1.1 200 OK
Content-Type: application/json

{"sensor_id":"SNS-5G-SEQ-000001","humidex_medio":29.87,"amostras":42}
===== CÓDIGO A.1.3b (resposta GET /indice-calor 200) — FIM =====
```

Resposta esperada, com HTTP 404, quando o sensor ainda não possui medição:

```
===== CÓDIGO A.1.3c (resposta GET /indice-calor 404) — INÍCIO =====
HTTP/1.1 404 Not Found
Content-Type: text/plain; charset=utf-8

Nenhuma medicao encontrada para o sensor
```

```
===== CÓDIGO A.1.3c (resposta GET /indice-calor 404) — FIM =====
```

O 404 é resposta válida do contrato e não conta como falha no gerador de carga.

A.1.4 Contagem de registros persistidos

```
===== CÓDIGO A.1.4 (curl GET /audit/contagem) — INÍCIO =====
curl -i http://localhost:8080/audit/contagem
===== CÓDIGO A.1.4 (curl GET /audit/contagem) — FIM =====
```

Resposta esperada, com HTTP 200:

```
===== CÓDIGO A.1.4b (resposta GET /audit/contagem) — INÍCIO =====
HTTP/1.1 200 OK
Content-Type: application/json

{"total_medicoes":8330,"total_indices":8330}
===== CÓDIGO A.1.4b (resposta GET /audit/contagem) — FIM =====
```

É a rota usada na drenagem: o avaliador repete a chamada até dois ciclos consecutivos devolverem os mesmos valores, por três leituras estáveis. Os dois números devem ser iguais; divergência é reportada como DIVERGENCIA_TABELAS.

A.1.5 Listagem paginada de medições

Primeira página, com limite explícito e cursor inicial:

```
===== CÓDIGO A.1.5 (curl GET /audit/medidas) — INÍCIO =====
curl -i "http://localhost:8080/audit/medidas?limit=2&cursor=0"
===== CÓDIGO A.1.5 (curl GET /audit/medidas) — FIM =====
```

Resposta esperada, com HTTP 200:

```
===== CÓDIGO A.1.5b (resposta GET /audit/medidas) — INÍCIO =====
HTTP/1.1 200 OK
Content-Type: application/json

{
  "medidas": [
    {
      "id": 1,
      "sensor_id": "SNS-5G-SEQ-000001",
      "timestamp": "2026-10-23T14:32:05.123456Z",
      "raw_packet": "k3j9x2ab7c1",
      "hash": "a3f1c9d84b7e2065f8a1d3c5e7b90246d1f3a5c7e9b1024368a5c7d9e1f3b5a7"
    },
    {
      "id": 2,
      "sensor_id": "SNS-5G-SEQ-000001",
      "timestamp": "2026-10-23T14:33:05.123456Z",
      "raw_packet": "m8p2v4cd9e5",
      "hash": "b7e2d1a94c6f3085a2b4d6e8f0c13579e2a4c6d8f0b2135479b6d8e0f2a4c6b8"
    }
  ],
  "proximo_cursor": "2"
}
===== CÓDIGO A.1.5b (resposta GET /audit/medidas) — FIM =====
```

Página final, quando o cursor devolvido é vazio:

```
===== CÓDIGO A.1.5c (resposta GET /audit/medidas — fim do conjunto) — INÍCIO =====
{
  "medidas": [],
  "proximo_cursor": ""
}
===== CÓDIGO A.1.5c (resposta GET /audit/medidas — fim do conjunto) — FIM =====
```

Percurso completo do conjunto, do jeito que o avaliador faz:

```
===== CÓDIGO A.1.5d (percurso paginado com curl) — INÍCIO =====
cursor=0
while :; do
  pagina=$(curl -s "http://localhost:8080/audit/medidas?limit=1000&cursor=$cursor")
  echo "$pagina"
  cursor=$(echo "$pagina" | grep -o '"proximo_cursor":"[^"]*"' | cut -d'"' -f4)
  [ -z "$cursor" ] && break
done
===== CÓDIGO A.1.5d (percurso paginado com curl) — FIM =====
```

O cursor é opaco: a plataforma não interpreta o valor, apenas o devolve na chamada seguinte.

A.1.6 Verificação manual do hash

Para conferir se o hash gravado corresponde à âncora do contrato:

```
===== CÓDIGO A.1.6 (verificação manual do hash) — INÍCIO =====
printf '%s' "SNS-5G-SEQ-0000012026-10-23T14:32:05.123456Zk3j9x2ab7c1" | sha256sum
===== CÓDIGO A.1.6 (verificação manual do hash) — FIM =====
```

A saída deve ser idêntica ao campo hash devolvido por GET /audit/medidas. A concatenação é direta, sem separadores, na ordem sensor_id, timestamp canônico e raw_packet.

ANEXO B — EXEMPLO DE SAÍDA DO AVALIADOR

O bloco reproduz a forma do log final. Os valores são ilustrativos.

```
RESULTADO DA AVALIACAO
==============================================
INGESTAO (POST /telemetria)
  Aceitas:        412  (98.80%)
  Perdidas:         5  (1.20%)
  Total:          417
CONSULTA (GET /indice-calor)
  Aceitas:       7810  (99.95%)
  Perdidas:         4  (0.05%)
  Total:         7814
  Sem dado:        92
REGISTROS
  Enviados:      8450
  Rejeitados:     110  (1.30%)
  Gravados:      8330  (98.58%)
==============================================
TAXA DE PERSISTENCIA: 98.58%  (gravados / enviados)
NOTA (proporcional):   9.86 / 10.00
==============================================
INTEGRIDADE (cruzamento manifesto x API)
  Divergentes: 0 | Fantasmas: 0 | Perdidas: 0 | Duplicatas: 0
  Coletadas: 8330 | Medicoes: 8330 | Indices: 8330
  Consistencia entre tabelas: OK
```

A linha resultado_registros.txt correspondente contém oito campos separados por ponto e vírgula, na ordem: enviados, rejeitados, gravados, taxa, divergentes, fantasmas, perdidas, duplicatas.

ANEXO C — LISTA DE VERIFICAÇÃO DE CONFORMIDADE

Antes da submissão, confirme cada item.

- A aplicação responde em http://aps-app:8080 dentro da rede aps-net, com o alias declarado no compose.
- O endpoint /health retorna 200 quando a aplicação está pronta.
- O endpoint /audit/contagem responde com total_medicoes e total_indices.
- O endpoint /audit/medidas responde com array medidas e campo proximo_cursor, e a iteração sucessiva do cursor alcança o conjunto completo.
- Cada item de medidas contém id, sensor_id, timestamp, raw_packet e hash.
- O timestamp é devolvido em UTC com seis casas decimais, no formato canônico.
- O hash corresponde a SHA-256 de sensor_id concatenado ao timestamp canônico concatenado a raw_packet, em hexadecimal minúsculo, sem separadores.
- Não existem dois registros com o mesmo sensor_id e o mesmo timestamp.
- Toda instância da aplicação declara cpus: '0.05'.
- A soma de limits.cpus de todos os containers não ultrapassa 0.80.
- A porta 8080 está publicada.
- Nenhum volume é declarado como external.
- O arquivo docker-compose.distribuido.yaml é válido segundo docker compose config.

ANEXO D — KIT BASE (CÓDIGOS FUNCIONAIS)

Os códigos a seguir são o kit estrutural do laboratório. Todos são funcionais e devem ser reproduzidos preservando integralmente suas estruturas e parâmetros. O arquivo init.sql, com a criação das tabelas medicoes e indices_calor usadas pelo main.go, é referenciado pelo docker-compose.monolito.yaml e deve acompanhar o kit na raiz do projeto.

D.1 main.go — Serviço monolítico de referência em Go

```
===== CÓDIGO D.1 (main.go) — INÍCIO =====
package main

import (
        "context"
        "crypto/sha256"
        "encoding/hex"
        "encoding/json"
        "fmt"
        "math"
        "net/http"
        "os"
        "strconv"
        "strings"
        "time"

        "github.com/jackc/pgx/v5/pgxpool"
)

var db *pgxpool.Pool

// ---------- Data contracts ----------
```

```
type Medicao struct {
	SensorID  string `json:"sensor_id"`
	Timestamp string `json:"timestamp"`
	Metrics   struct {
		Microclimate struct {
			TemperatureCelsius         float64 `json:"temperature_celsius"`
			RelativeHumidityPercentage float64 `json:"relative_humidity_percentage"`
		} `json:"microclimate"`
	} `json:"metrics"`
	RawTelemetryPacket string `json:"raw_telemetry_packet"`
}

type TelemetriaBatchRequest struct {
	Measurements []Medicao `json:"measurements"`
}

type TelemetriaBatchResponse struct {
	Status       string  `json:"status"`
	SensorID     string  `json:"sensor_id"`
	TotalAceito  int     `json:"total_aceito"`
	HumidexMedio float64 `json:"humidex_medio"`
}

// ---------- CPU-bound workload ----------
// A ancora de hash e PARTE DO CONTRATO de auditoria com o avaliador:
// SHA-256 unico sobre sensor_id + timestamp canonico + raw_packet, em hex.
// O timestamp entra em forma CANONICA (UTC, microssegundos fixos): a
// ingestao canonicaliza a string recebida e o endpoint /audit/medidas
// formata o valor gravado com o MESMO layout — assim o hash permanece
// verificavel conteudo a conteudo apos o round-trip no Postgres
// (timestamptz), independentemente do formato original do cliente.
const formatoHashTimestamp = "2006-01-02T15:04:05.000000Z07:00"

func timestampCanonico(s string) string {
	ts, err := time.Parse(time.RFC3339, s)
	if err != nil {
		return s // fallback: usa a string bruta (o k6 sempre envia RFC3339)
	}
	return ts.UTC().Format(formatoHashTimestamp)
}

func calcularHashMedicao(m *Medicao) string {
	entrada := m.SensorID + timestampCanonico(m.Timestamp) + m.RawTelemetryPacket
	sum := sha256.Sum256([]byte(entrada))
	return hex.EncodeToString(sum[:])
}

// calcularHumidex computes the microclimate heat index (non-linear, CPU-bound).
// e = actual vapor pressure (hPa); Humidex = T + 0.5555 * (e - 10).
func calcularHumidex(temperatura, umidade float64) float64 {
	e := 6.112 * math.Exp((17.62*temperatura)/(243.12+temperatura)) * (umidade / 100.0)
	return temperatura + 0.5555*(e-10.0)
}

// ---------- HTTP handlers ----------
func healthCheck(w http.ResponseWriter, r *http.Request) {
	ctx, cancel := context.WithTimeout(r.Context(), 2*time.Second)
	defer cancel()
	if err := db.Ping(ctx); err != nil {
		http.Error(w, "Banco de dados indisponivel", http.StatusServiceUnavailable)
		return
	}
	w.Header().Set("Content-Type", "application/json")
	json.NewEncoder(w).Encode(map[string]string{"status": "ok"})
}

func receberTelemetria(w http.ResponseWriter, r *http.Request) {
	var req TelemetriaBatchRequest
	if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
		http.Error(w, "Payload de telemetria invalido", http.StatusBadRequest)
		return
	}
	if len(req.Measurements) == 0 {
```

```
                http.Error(w, "Lote de medidas vazio", http.StatusBadRequest)
                return
        }
        ctx, cancel := context.WithTimeout(r.Context(), 5*time.Second)
        defer cancel()
        humidexes := make([]float64, len(req.Measurements))
        somaHumidex := 0.0
        for i := range req.Measurements {
                humidexes[i] = calcularHumidex(
                        req.Measurements[i].Metrics.Microclimate.TemperatureCelsius,
                        req.Measurements[i].Metrics.Microclimate.RelativeHumidityPercentage,
                )
                somaHumidex += humidexes[i]
        }
        humidexMedio := somaHumidex / float64(len(req.Measurements))

        // Single transaction for the whole batch: one commit, one WAL fsync.
        // pgx v5 caches prepared statements automatically (default
        // QueryExecModeCacheStatement), so direct calls in the loop avoid
        // re-parsing without any explicit Prepare.
        tx, err := db.Begin(ctx)
        if err != nil {
                http.Error(w, "Falha ao abrir transacao", http.StatusInternalServerError)
                return
        }
        defer tx.Rollback(ctx)

        for i := range req.Measurements {
                m := &req.Measurements[i]
                var medicaoID int64
                if err := tx.QueryRow(ctx,
                        `INSERT INTO medicoes (sensor_id, criado_em, temperatura, umidade, raw_packet,
hash_pacote)
                         VALUES ($1, $2::timestamptz, $3, $4, $5, $6) RETURNING id`,
                        m.SensorID, m.Timestamp,
                        m.Metrics.Microclimate.TemperatureCelsius,
                        m.Metrics.Microclimate.RelativeHumidityPercentage,
                        m.RawTelemetryPacket,
                        calcularHashMedicao(m),
                ).Scan(&medicaoID); err != nil {
                        http.Error(w, "Falha ao gravar medicao", http.StatusInternalServerError)
                        return
                }
                if _, err := tx.Exec(ctx,
                        `INSERT INTO indices_calor (sensor_id, humidex, medicao_id) VALUES ($1, $2,
$3)`,
                        m.SensorID, humidexes[i], medicaoID,
                ); err != nil {
                        http.Error(w, "Falha ao gravar indice", http.StatusInternalServerError)
                        return
                }
        }
        if err := tx.Commit(ctx); err != nil {
                http.Error(w, "Falha ao confirmar lote", http.StatusInternalServerError)
                return
        }

        w.Header().Set("Content-Type", "application/json")
        w.WriteHeader(http.StatusAccepted)
        json.NewEncoder(w).Encode(TelemetriaBatchResponse{
                Status:       "aceito",
                SensorID:     req.Measurements[0].SensorID,
                TotalAceito:  len(req.Measurements),
                HumidexMedio: humidexMedio,
        })
}

func obterIndiceMedio(w http.ResponseWriter, r *http.Request) {
        sensorID := strings.TrimPrefix(r.URL.Path, "/indice-calor/")
        if sensorID == "" {
                http.Error(w, "sensor_id obrigatorio", http.StatusBadRequest)
                return
        }
        ctx, cancel := context.WithTimeout(r.Context(), 50*time.Second)
        defer cancel()
```

```
        var media *float64
        var amostras int64
        err := db.QueryRow(ctx,
                `SELECT AVG(humidex), COUNT(*) FROM indices_calor WHERE sensor_id = $1`,
                sensorID,
        ).Scan(&media, &amostras)
        if err != nil {
                http.Error(w, "Falha ao consultar indice", http.StatusInternalServerError)
                return
        }
        if media == nil {
                http.Error(w, "Nenhuma medicao encontrada para o sensor", http.StatusNotFound)
                return
        }
        w.Header().Set("Content-Type", "application/json")
        json.NewEncoder(w).Encode(map[string]interface{}{
                "sensor_id":     sensorID,
                "humidex_medio": *media,
                "amostras":      amostras,
        })
}

// ---------- Contrato de auditoria (exigido pelo avaliador) ----------
// O avaliador NAO acessa o banco diretamente: quem gerencia o acesso ao
// banco e esta API. Alunos podem mudar o schema livremente, desde que
// estes endpoints continuem respondendo no formato contratado:
//
//   GET /audit/contagem               -> {"total_medicoes": N, "total_indices": M}
//   GET /audit/medidas?cursor=&limit= -> {"medidas":[{id,sensor_id,timestamp,raw_packet,hash}...],
//                                         "proximo_cursor":"<id ou vazio>"}
func auditoriaContagem(w http.ResponseWriter, r *http.Request) {
        ctx, cancel := context.WithTimeout(r.Context(), 5*time.Second)
        defer cancel()
        var totalMed, totalInd int64
        err := db.QueryRow(ctx,
                `SELECT (SELECT COUNT(*) FROM medicoes), (SELECT COUNT(*) FROM indices_calor)`,
        ).Scan(&totalMed, &totalInd)
        if err != nil {
                http.Error(w, "erro de auditoria", http.StatusInternalServerError)
                return
        }
        w.Header().Set("Content-Type", "application/json")
        json.NewEncoder(w).Encode(map[string]int64{
                "total_medicoes": totalMed,
                "total_indices":  totalInd,
        })
}

func auditoriaMedidas(w http.ResponseWriter, r *http.Request) {
        limit := 1000
        if v, err := strconv.Atoi(r.URL.Query().Get("limit")); err == nil && v > 0 && v <= 5000 {
                limit = v
        }
        cursor, _ := strconv.ParseInt(r.URL.Query().Get("cursor"), 10, 64)
        ctx, cancel := context.WithTimeout(r.Context(), 30*time.Second)
        defer cancel()
        rows, err := db.Query(ctx,
                `SELECT id, sensor_id, criado_em, raw_packet, hash_pacote
                   FROM medicoes WHERE id > $1 ORDER BY id LIMIT $2`, cursor, limit)
        if err != nil {
                http.Error(w, "erro de auditoria", http.StatusInternalServerError)
                return
        }
        defer rows.Close()
        type medidaAudit struct {
                ID        int64  `json:"id"`
                SensorID  string `json:"sensor_id"`
                Timestamp string `json:"timestamp"`
                RawPacket string `json:"raw_packet"`
                Hash      string `json:"hash"`
        }
        medidas := []medidaAudit{}
        proximo := ""
        var ultimoID int64
        for rows.Next() {
                var m medidaAudit
```

```
                var criadoEm time.Time
                if err := rows.Scan(&m.ID, &m.SensorID, &criadoEm, &m.RawPacket, &m.Hash); err != nil {
                        http.Error(w, "erro de auditoria", http.StatusInternalServerError)
                        return
                }
                m.Timestamp = criadoEm.UTC().Format(formatoHashTimestamp)
                ultimoID = m.ID
                medidas = append(medidas, m)
        }
        if err := rows.Err(); err != nil {
                http.Error(w, "erro de auditoria", http.StatusInternalServerError)
                return
        }
        if len(medidas) == limit {
                proximo = strconv.FormatInt(ultimoID, 10)
        }
        w.Header().Set("Content-Type", "application/json")
        json.NewEncoder(w).Encode(map[string]interface{}{
                "medidas":        medidas,
                "proximo_cursor": proximo,
        })
}

// ---------- Bootstrap ----------
func main() {
        dsn := os.Getenv("DATABASE_URL")
        if dsn == "" {
                panic("DATABASE_URL nao definida")
        }
        // Frente 1: bounded connection pool. The Postgres never sees more than
        // MaxConns simultaneous operations; the waiting queue lives inside the
        // application, where the distributed techniques (cache, queue) will act.
        config, err := pgxpool.ParseConfig(dsn)
        if err != nil {
                panic(err)
        }
        config.MaxConns = 10
        config.MinConns = 1
        db, err = pgxpool.NewWithConfig(context.Background(), config)
        if err != nil {
                panic(err)
        }
        if err = db.Ping(context.Background()); err != nil {
                panic(err)
        }
        http.HandleFunc("/health", healthCheck)
        http.HandleFunc("/telemetria", receberTelemetria)
        http.HandleFunc("/indice-calor/", obterIndiceMedio)
        http.HandleFunc("/audit/contagem", auditoriaContagem)
        http.HandleFunc("/audit/medidas", auditoriaMedidas)
        fmt.Println("Servidor Go inicializado com sucesso na porta 8080...")
        if err := http.ListenAndServe(":8080", nil); err != nil {
                panic(err)
        }
}
===== CÓDIGO D.1 (main.go) — FIM =====
```

D.4 Dockerfile.monolito — Compilação multi-stage da aplicação Go

```
===== CÓDIGO D.4 (Dockerfile.monolito) — INÍCIO =====
FROM golang:1.25-alpine AS builder
WORKDIR /app
COPY main.go .
RUN go mod init monitoramento/monolito && \
    go get github.com/jackc/pgx/v5 && \
    go mod tidy && \
    CGO_ENABLED=0 GOOS=linux go build -a -installsuffix cgo -o monolito main.go

FROM alpine:latest
RUN apk --no-cache add ca-certificates
WORKDIR /root/
COPY --from=builder /app/monolito .
EXPOSE 8080
CMD ["./monolito"]
===== CÓDIGO D.4 (Dockerfile.monolito) — FIM =====
```

D.6 docker-compose.monolito.yaml — Sistema sob teste (baseline do Cenário A)

```
===== CÓDIGO D.6 (docker-compose.monolito.yaml) — INÍCIO =====
name: aps-sistema


# docker-compose.monolito.yaml — SISTEMA SOB TESTE (baseline Cenario A)
# Toda a soma de limits.cpus da solucao vive NESTE arquivo (auditoria do orcamento).
# Contrato com a plataforma de teste: o alias aps-app na rede externa aps-net.
# O banco fica na rede interna do compose — inacessivel para a plataforma de
# teste: quem gerencia o acesso ao banco e a API da aplicacao (endpoints /audit/*).
services:
  postgres:
    image: postgres:16-alpine
    environment:
      - POSTGRES_USER=iot
      - POSTGRES_PASSWORD=iot
      - POSTGRES_DB=telemetria
    deploy:
      resources:
        limits:
          cpus: '0.08'
          memory: 256M
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U iot -d telemetria"]
      interval: 5s
      timeout: 5s
      retries: 20
      start_period: 15s


  db-init:
    image: postgres:16-alpine
    environment:
      - PGPASSWORD=iot
    volumes:
      - ./init.sql:/init.sql:ro
    entrypoint: ["/bin/sh", "-c"]
    command:
      - |
        for i in $$(seq 1 30); do
          psql -h postgres -U iot -d telemetria -f /init.sql && exit 0
          sleep 1
        done
        echo "Falha ao aplicar init.sql" && exit 1
    depends_on:
      postgres:
        condition: service_healthy


  sistema-monolito:
    build:
      context: .
      dockerfile: Dockerfile.monolito
    environment:
      - DATABASE_URL=postgres://iot:iot@postgres:5432/telemetria?sslmode=disable
      - GOMAXPROCS=2
    deploy:
      replicas: 1
      resources:
        limits:
          cpus: '0.05'
          memory: 64M
    ports:
      - "8080:8080"
    networks:
      default:
      aps-net:
        aliases: [aps-app]
    healthcheck:
      test: ["CMD-SHELL", "wget -qO- http://localhost:8080/health || exit 1"]
      interval: 1s
      timeout: 2s
      retries: 30
      start_period: 10s
```

```
    depends_on:
      db-init:
        condition: service_completed_successfully
      postgres:
        condition: service_healthy

networks:
  aps-net:
    external: true
===== CÓDIGO D.6 (docker-compose.monolito.yaml) — FIM =====
```

D.7 docker-compose.teste.yaml — Plataforma de teste (instrumento de medição)

```
===== CÓDIGO D.7 (docker-compose.teste.yaml) — INÍCIO =====
name: aps-teste

# docker-compose.teste.yaml — PLATAFORMA DE TESTE (instrumento de medicao)
# Imutavel: wait-app + k6 + avaliador. Sem cota de CPU (instrumento, fora do
# orcamento 0.80). Contrato: fala apenas com o alias aps-app:8080 na rede
# externa aps-net — nunca com o banco diretamente (auditoria via API).
services:
  wait-app:
    image: alpine:3.20
    entrypoint: ["/bin/sh", "-c"]
    command:
      - |
        echo "Aguardando a aplicacao em http://aps-app:8080/health ..."
        for i in $$(seq 1 60); do
          if wget -qO- http://aps-app:8080/health >/dev/null 2>&1; then
            echo "Aplicacao pronta."
            exit 0
          fi
          sleep 2
        done
        echo "Aplicacao nao ficou pronta a tempo."
        exit 1
    networks: [aps-net]

  stress-test:
    image: grafana/k6:latest
    environment:
      BASE_URL: http://aps-app:8080
      PEAK_START: ${PEAK_START:-30}
      READ_RATE: ${READ_RATE:-50}
      WRITE_RATE: ${WRITE_RATE:-4}
      PEAK_FACTOR: ${PEAK_FACTOR:-3}
      BATCH_MIN: ${BATCH_MIN:-10}
      BATCH_MAX: ${BATCH_MAX:-60}
      READ_POOL_RATIO: ${READ_POOL_RATIO:-0.6}
    volumes:
      - .:/workspace
    working_dir: /workspace
    entrypoint: ["/bin/sh", "-c"]
    command:
      - |
        mkdir -p logs results
        k6 run --summary-export=logs/report.json --out json=logs/stream.json script.js
    networks: [aps-net]

  avaliador-notas:
    image: golang:1.25-alpine
    environment:
      - API_BASE_URL=http://aps-app:8080
      - BASE_DIR=/app
    volumes:
      - .:/app
      - go-mod-cache:/go/pkg/mod
    working_dir: /app/avaliador
    entrypoint: ["/bin/sh", "-c"]
    command:
      - |
        go mod init avaliador 2>/dev/null || true
```

```
        go run /app/avaliar_justo.go
    networks: [aps-net]

volumes:
  go-mod-cache:

networks:
  aps-net:
    external: true
===== CÓDIGO D.7 (docker-compose.teste.yaml) — FIM =====
```

D.8 run_benchmark.bat — Orquestrador da APS (Windows CMD)

```
===== CÓDIGO D.8 (run_benchmark.bat) — INÍCIO =====
@echo off
setlocal
pushd "%~dp0"
set "BENCH_EXIT=1"

REM ============================================================
REM run_benchmark.bat - Orquestrador da APS (Windows CMD)
REM Uso:
REM   run_benchmark.bat                                    (monolito padrao)
REM   run_benchmark.bat docker-compose.distribuido.yaml    (solucao do aluno)
REM   set APP_COMPOSE=docker-compose.distribuido.yaml && run_benchmark.bat
REM ============================================================

if not "%~1"=="" set "APP_COMPOSE=%~1"
if not defined APP_COMPOSE set "APP_COMPOSE=docker-compose.monolito.yaml"

echo ================================================
echo APS Benchmark - sistema sob teste: %APP_COMPOSE%
echo ================================================

docker info >nul 2>&1
if errorlevel 1 (
    echo ERRO: Docker nao disponivel. Abra o Docker Desktop e tente novamente.
    exit /b 1
)

if not exist "%APP_COMPOSE%" (
    echo ERRO: arquivo %APP_COMPOSE% nao encontrado nesta pasta.
    exit /b 1
)

if not exist "docker-compose.teste.yaml" (
    echo ERRO: arquivo docker-compose.teste.yaml nao encontrado nesta pasta.
    exit /b 1
)

docker network create aps-net >nul 2>&1

echo Subindo o sistema sob teste...
docker compose -f "%APP_COMPOSE%" up -d --build
if errorlevel 1 (
    echo ERRO: falha ao subir o sistema sob teste.
    goto :limpar
)

echo Aguardando a aplicacao ficar saudavel...
docker compose -f docker-compose.teste.yaml run --rm wait-app
if errorlevel 1 (
    echo ERRO: aplicacao nao ficou pronta a tempo.
    goto :limpar
)

echo Executando o teste de carga k6...
docker compose -f docker-compose.teste.yaml run --rm stress-test
if errorlevel 1 (
    echo ERRO: falha na execucao do k6.
```

```
    goto :limpar
)

REM Delay de sincronizacao: alunos podem usar Master/Slave ou mensageria.
REM Aguarda o sistema terminar de sincronizar antes da checagem final.
echo Aguardando 15s para sincronizacao de dados (replicacao/mensageria)...
timeout /t 15 /nobreak >nul

echo Executando o avaliador...
docker compose -f docker-compose.teste.yaml run --rm avaliador-notas
set "BENCH_EXIT=%errorlevel%"

:limpar
echo Encerrando e limpando containers e volumes...
docker compose -f "%APP_COMPOSE%" down -v
docker compose -f docker-compose.teste.yaml down

echo.
echo Artefatos: logs\ report.json e stream.json - results\ nota, integridade e curvas

popd
exit /b %BENCH_EXIT%
===== CÓDIGO D.8 (run_benchmark.bat) — FIM =====
```

D.9 run_benchmark.sh — Orquestrador da APS (Linux/macOS)

```
===== CÓDIGO D.9 (run_benchmark.sh) — INÍCIO =====
#!/bin/sh
# ============================================================
# run_benchmark.sh - Orquestrador da APS (Linux/macOS)
# Uso:
#   ./run_benchmark.sh                                    (monolito padrao)
#   ./run_benchmark.sh docker-compose.distribuido.yaml    (solucao do aluno)
#   APP_COMPOSE=docker-compose.distribuido.yaml ./run_benchmark.sh
# ============================================================
set -u

APP_COMPOSE="${1:-docker-compose.monolito.yaml}"
BENCH_EXIT=1

cleanup() {
    echo ""
    echo "Encerrando e limpando containers e volumes..."
    docker compose -f "$APP_COMPOSE" down -v
    docker compose -f docker-compose.teste.yaml down
}

cd "$(dirname "$0")" || exit 1

echo "============================================="
echo "APS Benchmark - sistema sob teste: $APP_COMPOSE"
echo "============================================="

# 0. Docker disponivel?
if ! docker info >/dev/null 2>&1; then
    echo "ERRO: Docker nao disponivel. Inicie o servico do Docker e tente novamente."
    exit 1
fi

# Validacao de arquivos (falha rapida antes de subir qualquer container)
if [ ! -f "$APP_COMPOSE" ]; then
    echo "ERRO: arquivo $APP_COMPOSE nao encontrado nesta pasta."
    exit 1
fi

if [ ! -f "docker-compose.teste.yaml" ]; then
    echo "ERRO: arquivo docker-compose.teste.yaml nao encontrado nesta pasta."
    exit 1
fi
```

```
# A partir daqui, qualquer saida passa pela limpeza (inclusive Ctrl+C)
trap cleanup EXIT
trap 'exit 130' INT
trap 'exit 143' TERM


# 1. Rede compartilhada (idempotente)
docker network create aps-net >/dev/null 2>&1 || true


# 2. Sobe o sistema sob teste (monolito de referencia ou a solucao do aluno)
echo "Subindo o sistema sob teste..."
if ! docker compose -f "$APP_COMPOSE" up -d --build; then
    echo "ERRO: falha ao subir o sistema sob teste."
    exit 1
fi


# 3. Plataforma de teste, etapa por etapa (run --rm: exit code real por etapa)
echo "Aguardando a aplicacao ficar saudavel..."
if ! docker compose -f docker-compose.teste.yaml run --rm wait-app; then
    echo "ERRO: aplicacao nao ficou pronta a tempo."
    exit 1
fi


echo "Executando o teste de carga k6..."
if ! docker compose -f docker-compose.teste.yaml run --rm stress-test; then
    echo "ERRO: falha na execucao do k6."
    exit 1
fi


# Delay de sincronizacao: alunos podem usar Master/Slave ou mensageria.
# Aguarda o sistema terminar de sincronizar antes da checagem final.
echo "Aguardando 15s para sincronizacao de dados (replicacao/mensageria)..."
sleep 15


echo "Executando o avaliador..."
if docker compose -f docker-compose.teste.yaml run --rm avaliador-notas; then
    BENCH_EXIT=0
    echo "Benchmark concluido com sucesso."
else
    echo "Benchmark concluido com falhas. Verifique os logs acima."
fi


echo ""
echo "Artefatos: logs/ (report.json e stream.json) - results/ (nota, integridade e curvas)"
exit $BENCH_EXIT
===== CÓDIGO D.9 (run_benchmark.sh) — FIM =====
```